

Music Playlist Manager

Documentazione dell'Architettura Software

Gruppo 10

Software Architecture Design — Università degli Studi di Salerno

Sintesi. *Music Playlist Manager* è un'applicazione desktop per gestire una libreria musicale e playlist, con riproduttore integrato (play/pausa, avanti/indietro, shuffle, loop), annullamento delle operazioni e classifica dei brani più ascoltati. L'architettura adotta il pattern **MVC** affiancato da un **livello di Service**, e poggia su sei design pattern **GoF**: Singleton, Command, State, Strategy, Observer, Iterator.

Indice

1	Introduzione	2
2	Visione architetturale d'insieme	2
3	MVC e livello di Service	2
4	Design pattern (GoF)	3
5	Modello di dominio	5
6	Test ed esecuzione	5

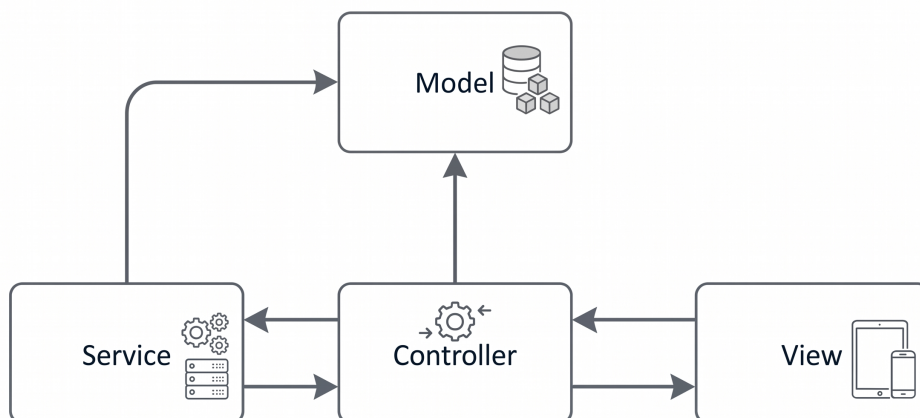
1 Introduzione

Aspetto	Tecnologia
Linguaggio	Java 17
Interfaccia	JavaFX 21
Build	Apache Maven
Test	JUnit 5 (21 classi di test)

All'avvio l'applicazione crea la finestra principale, carica la schermata della libreria e installa il livello di navigazione; da lì l'utente accede a tracce, playlist e player.

2 Visione architetturale d'insieme

L'architettura del sistema si fonda su una rigorosa applicazione del pattern **MVC (Model-View-Controller)**, affiancata da un livello di **Service applicativi** per le funzionalità trasversali. I Controller coordinano le interazioni dell'utente, delegando la gestione dello stato globale ai Service, mentre la logica di business pura e i dati sono solidamente incapsulati nel Modello di Dominio. L'intera infrastruttura è irrobustita dall'uso strategico di design pattern comportamentali e creazionali.



Package	Responsabilità
app	Avvio dell'applicazione.
track, collections.*	Modello e controller: Track , TrackCollection , Playlist , TrackList , PlaylistList .
player.*	Logica di riproduzione (Player), controller, state e strategy.
core.command, *.command	Pattern Command e CommandInvoker (undo).
core.observer, core.iterator	Pattern Observer e Iterator.
core.*, player.service	Service: navigazione, ciclo di vita player, classifiche.
tag, core.alert	Tag personali e finestre di dialogo.

3 MVC e livello di Service

MVC. La presentazione separa tre responsabilità:

- **Model** — incapsula le regole di business e lo stato dell'applicazione in modo del tutto indipendente dall'interfaccia grafica. Comprende le entità di dominio centrali come **Track** (che

modella i metadati musicali), l'astrazione `TrackCollection` con le sue implementazioni concrete `Playlist` e `TrackList` (che gestiscono raggruppamenti di brani), e il nucleo musicale `Player`. Il Modello include anche lo stato applicativo globale, rappresentato dallo storico annullabile gestito da `CommandInvoker` e dall'anagrafica centralizzata in `PersonalTagManager`;

- **View** — le schermate dichiarative e gli stili;
- **Controller** — le classi `*Controller` che ricevono gli eventi utente, coordinano modello e service e aggiornano la vista.

Il framework grafico collega automaticamente vista e dati, perciò i controller restano sottili e privi di logica di dominio.

Livello di Service. Incapsula le funzionalità trasversali e le risorse condivise, mantenendo i controller leggeri ed evitando duplicazioni:

- `NavigationManager` — cambio di schermata a finestra singola, con il player che rimane visibile come sovrapposizione persistente;
- `ActivePlayerManager` — apertura/chiusura e ciclo di vita del mini-player;
- `RankingService<T>` — classifica dei brani/playlist più riprodotti, aggiornata automaticamente;
- `TagUIHelper` — generazione e manipolazione dinamica dell'interfaccia per i tag;
- `AlertManager` — messaggi e richieste di conferma.

Ogni Service è un punto d'accesso unico (Singleton), così l'intera applicazione condivide lo stesso stato di navigazione, riproduzione e cronologia comandi.

4 Design pattern (GoF)

Per ciascun pattern si indicano l'*intento* e la mappatura dei *ruoli* sulle classi reali.

1. Singleton. *Intento:* un'unica istanza globale con accesso centralizzato. *Realizzazione:* inizializzazione immediata (`static final`) in `NavigationManager` e `ActivePlayerManager`; “a puntatore” (`exists()` + `getXPointer()`) in `TrackList`, `PlaylistList`, `CommandInvoker`; lazy classico in `PersonalTagManager`.

```
public static PersonalTagManager getInstance() {
    if (instance == null) {
        instance = new PersonalTagManager();
    }
    return instance;
}
```

2. Command. *Intento:* incapsulare un'operazione in un oggetto, per eseguirla, accodarla e **annullarla** in modo uniforme. *Ruoli:* `AbstractCommand` (*Command*, con `execute/undo`); `AddTrackCommand`, `RemoveTrackCommand`, `AddPlaylistCommand`, `DeletePlaylistCommand` (*Concrete-Command*, sulla gerarchia `BaseCommands`); `CommandInvoker` (*Invoker*); i controller (*Client*). Lo storico è una coda limitata a 10 comandi: `setCommand` esegue e memorizza, `undoCommand` annulla l'ultimo.

```
public void setCommand(AbstractCommand command){
    if (commands.size() >= SIZE_LIMIT) commands.removeFirst(); // scarta il piu'
    vecchio
    command.execute();
    commands.addLast(command);
}
```

```

}
public void undoCommand(){
    commands.removeLast().undo();    // annulla l'ultimo comando eseguito
}

```

3. State. *Intento:* variare il comportamento di un oggetto al variare del suo stato interno. *Ruoli:* Player (*Context*), PlayerState (*State*), Play e Pause (*ConcreteState*). Ogni stato esegue la propria azione e imposta lo stato successivo, spostando fuori dal controller la logica del passaggio play/pausa.

```

public void changeState() { currentState.execute(this); }    // Player delega allo stato

public void execute(Player player) {                        // stato Play
    player.startPlayback();
    player.setState(new Pause());                          // transizione a Pause
}

```

4. Strategy. *Intento:* famiglia di algoritmi intercambiabili a runtime — qui la *modalità di riproduzione*. *Ruoli:* ExecutionStrategy (*Strategy*), Sequential, Shuffle, Loop (*ConcreteStrategy*), Iterator (*Context*). Ogni strategia produce l'ordine di riproduzione (un array di indici): scorrimento sequenziale, casuale, oppure ripetizione del brano corrente. Shuffle e loop sono mutuamente esclusivi; in assenza di entrambi si usa Sequential.

```

public void setStrategy(ExecutionStrategy es) {
    this.currentStrategy = es;
    // Genera il nuovo ordine di riproduzione dinamico
    this.iterationindex = es.execute(tracks.size(), realCurrentIndex);
}

```

5. Observer. *Intento:* notificare automaticamente gli oggetti dipendenti al variare di un soggetto. *Ruoli:* BaseSubject/TrackList (*Subject*), BaseObserver/Playlist (*Observer*). Quando una traccia viene eliminata dalla libreria, TrackList notifica gli osservatori e ogni playlist che la contiene la rimuove da sé, senza che il chiamante debba conoscerle. La registrazione degli osservatori è gestita dai comandi su playlist (integrazione Command + Observer).

```

public void notifyObservers(Track track) {
    for(BaseObserver o: observers)
        o.update(track);    // Ogni Playlist si aggiorna rimuovendo la traccia
}

```

6. Iterator. *Intento:* scorrere una collezione senza esporne la struttura interna. *Ruoli:* AbstractIterator/Iterator con IterableCollection (creazione via *Factory Method* createIterator()); TrackCollection è l'aggregato concreto. Lo scorrimento è circolare e l'ordine effettivo è deciso dalla Strategy attiva.

```

public Track getNext() {
    // Se siamo arrivati alla fine, ricomincia da capo (comportamento circolare)
    if (currentindex < iterationindex.length - 1) currentindex++;
    else currentindex = 0;
    return tracks.get(iterationindex[currentindex]);
}

```

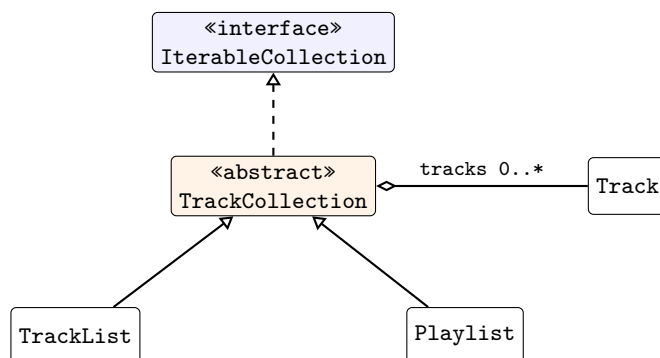
Altri idiomi. AlertManager è una *utility class* (costruttore privato, soli metodi statici); l'interfaccia MostPlayed, implementata da Track e Playlist, permette a RankingService<T extends MostPlayed> di riusare la stessa logica di classifica per entrambi.

5 Modello di dominio

La radice del dominio è l'astrazione `TrackCollection`, che incapsula la logica comune di aggregazione dei brani (possiede una lista osservabile di `Track` e fornisce metodi base per l'aggiunta, la rimozione e l'interrogazione). Da questa classe astratta derivano due implementazioni concrete distinte:

- `TrackList`: rappresenta la libreria globale dell'applicazione. È concepita come un Singleton (essendo unica per l'intero ciclo di vita del software) e agisce da *Subject* nel pattern Observer: quando un brano viene eliminato in modo definitivo dal sistema, `TrackList` notifica tutte le playlist iscritte affinché rimuovano anch'esse il brano per mantenere la coerenza.
- `Playlist`: rappresenta un raggruppamento logico personalizzato creato dall'utente. Agisce da *Observer* rispetto alla `TrackList` e mantiene un contatore indipendente di riproduzioni per monitorare le preferenze di ascolto.

Al livello atomico troviamo la classe `Track`, che modella dettagliatamente i metadati musicali (titolo, autore, anno, durata, tag). Infine, sia `Track` che `Playlist` implementano l'interfaccia `MostPlayed`, offrendo un contratto comune per il conteggio degli ascolti, che viene poi agevolmente processato dal `RankingService` per generare le classifiche.



6 Test ed esecuzione

21 classi JUnit 5 coprono modello e pattern (comandi, iteratore, strategie, stati, classifica). Comandi Maven principali: `mvn test` (test), `mvn javafx:run` (avvio), `mvn package` (produzione del JAR eseguibile).